OS_ASSIGNMENT9
2402MC07

Executive Summary & Implementation Details

Question 1: Printing the Page Table (vmprint)

- **Changes Made**: Added system call sys_vmprint() in sysproc.c and implementation vmprint() in vm.c.
- **Mechanism**: Traversing the page directory entries (pgdir) and checking for the PTE_P (present) bit. For valid page tables, page entries are checked up to KERNBASE to filter user-space mappings. The Virtual Address (VA), Physical Address (PA), and P, W, U flag bits are output for every valid user page.
- **Output Analysis**: The test program pttest.c shows an initial count of 3 valid user pages. After requesting memory via sbrk(4 * 4096) and populating them, 4 additional valid pages appear, bringing the total to 7 pages.

Question 2: Handling Page Faults / Lazy Allocation

- **Changes Made**:
  - sysproc.c: Updated sys_sbrk() so that positive requests only increment myproc()->sz without calling allocuvm().
  - vm.c: Updated deallocuvm() and copyuvm() to bypass pages where physical memory has not been mapped (*pte & PTE_P is false), preventing kernel panics.
  - trap.c: Added handling for trap type T_PGFLT. When a fault occurs below myproc()->sz, a physical page is allocated with kalloc(), zeroed, mapped using mappages(), and myproc()->page_faults is incremented. Any out-of-bounds access kills the process.
- **Output Analysis**: faulttest.c lazily requests 32 pages and writes to 5 distinct pages, triggering exactly 5 page faults. An out-of-bounds access at address 0x24000 triggers a page fault that is out of range, terminating the process as designed.

Question 3: Page Replacement Algorithms (FIFO vs. LRU)

- **Changes Made**: Created pagerepl.c to simulate FIFO and LRU page replacement policies given reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 7 0 1.
- **Results Table**:

Frame Count	FIFO Faults	FIFO Hit Ratio	LRU Faults	LRU Hit Ratio
3	15	21%	12	36%
4	10	47%	8	57%

5	7	63%	5	73%

- **Analysis**:
  - For 3 frames, FIFO results in 15 page faults while LRU results in 12, verifying the benchmark requirements.
  - LRU consistently outperforms FIFO across all frame sizes due to temporal locality, reducing total page faults.